

VIT BHOPAL UNIVERSITY

School of Computer Science and Artificial Intelligence (SCAI)



PROJECT REPORT

MATH TOOLKIT

A Modular, Menu-Driven Python Application for
Number Theory, Sequences and Conversions

VITyarthi - Build Your Own Project

Student Name	Mradanshu Sharma
Registration No.	26BAI10712
School	School of Computer Science and Artificial Intelligence (SCAI)
University	VIT Bhopal University
Course Name	Introduction to Problem Solving and Programming
Session	2026 - 2030
Language / Tools	Python 3, unittest, Git & GitHub

Table of Contents

No.	Section
1	Introduction
2	Problem Statement
3	Functional Requirements
4	Non-functional Requirements
5	System Architecture
6	Design Diagrams
7	Design Decisions & Rationale
8	Implementation Details
9	Screenshots / Results
10	Testing Approach
11	Challenges Faced
12	Learnings & Key Takeaways
13	Future Enhancements
14	References

The cover page is the first page of this report; the sections above follow the order required by the VITyarthi submission guidelines.

1. Introduction

Programming is learned best by solving small, well-defined problems. In the first semester, students repeatedly write programs to check prime numbers, find factors, compute the GCD, print the Fibonacci series or convert decimal numbers to binary. These programs are usually written as separate files and are never combined into one usable tool.

Math Toolkit brings all of these together into a single console application. The user chooses one of 15 menu options, enters the required numbers, and receives the answer instantly. A special **Full Analysis** option combines several functions to produce a complete report about a number.

The project is designed to demonstrate the core concepts of the course Introduction to Problem Solving and Programming: variables and data types, conditional statements, loops, functions, lists, string handling, modular programming, input validation, error handling and testing.

Purpose of this report: to document the problem, requirements, design, implementation, testing and learnings of the project as required by the VITyarthi evaluation guidelines.

2. Problem Statement

Beginners solve the same basic number problems again and again as isolated programs. There is no single, simple and crash-proof tool that (a) gathers these problems in one place, (b) lets a student verify answers quickly, (c) handles wrong input politely, and (d) shows how small functions combine into a larger program.

Objectives

- Build one menu-driven application that provides at least 14 mathematical utilities.
- Apply modular programming - split the code into at least 5-10 meaningful files/modules.
- Validate every input so that wrong data never crashes the program.
- Write unit tests for every core function and verify the results automatically.
- Maintain the code in Git and document it with README, statement and this report.

Scope and Target Users

In scope: console application, integer/float number utilities, validation, testing. **Out of scope:** GUI, database, network features, very large-number performance. **Target users:** first-year engineering students, teachers demonstrating algorithms, and anyone who needs quick number checks.

3. Functional Requirements

The project provides five major functional modules (the guideline requires at least three), a clear input/output structure and a logical workflow.

ID	Module	Feature	Input	Output
FR1	Number Theory	Check Prime	integer	prime / not prime message
FR2	Number Theory	Find Factors	integer ≥ 1	list of all factors
FR3	Number Theory	Prime Factorization	integer ≥ 2	list of prime factors
FR4	Number Theory	Find GCD	two integers	GCD value
FR5	Number Theory	Smallest Divisor	integer ≥ 1	smallest divisor > 1
FR6	Sequences	Factorial	integer ≥ 0	$n!$
FR7	Sequences	Fibonacci	number of terms	list of terms
FR8	Converters	Reverse Number	integer ≥ 0	reversed number
FR9	Converters	Decimal to Binary	integer ≥ 0	binary string
FR10	Converters	Char to Number	one character	ASCII value
FR11	Math Operations	Square Root	number ≥ 0	square root
FR12	Math Operations	Power	base, exponent	base^{exp}
FR13	Math Operations	Random Number	start, end	random integer in range
FR14	Full Analysis	Full Analysis	integer ≥ 1	even/odd, prime, divisor, factors, reverse, binary
FR15	Menu	Exit	-	program closes

4. Non-functional Requirements

ID	Category	Requirement	How it is achieved
NFR1	Usability	Simple numbered menu, clear prompts and readable output.	Single menu screen; every prompt states what to enter.
NFR2	Reliability	The program must never crash on wrong input.	validators.py catches ValueError and re-asks until input is valid.
NFR3	Maintainability	Code must be easy to read, change and extend.	7 separate modules, docstrings on every function, one job per function.
NFR4	Performance	Results appear instantly for normal input sizes.	Prime check runs only up to the square root of n ; Euclidean algorithm for GCD.
NFR5	Error handling	Invalid choices/ranges give friendly messages.	Range checks (minimum values) + 'Wrong choice' message in the menu.
NFR6	Portability	Runs on any OS with Python 3.8+.	Uses only the standard library - no installation needed.
NFR7	Testability	Every core function is verifiable automatically.	17 unittest cases including mocked user input.

5. System Architecture

The application follows a simple **layered architecture**. The user talks only to the presentation layer (main.py and menu.py). The menu dispatches each choice to a feature module in the logic layer. All keyboard input passes through validators.py in the support layer, so the logic modules always receive clean data.

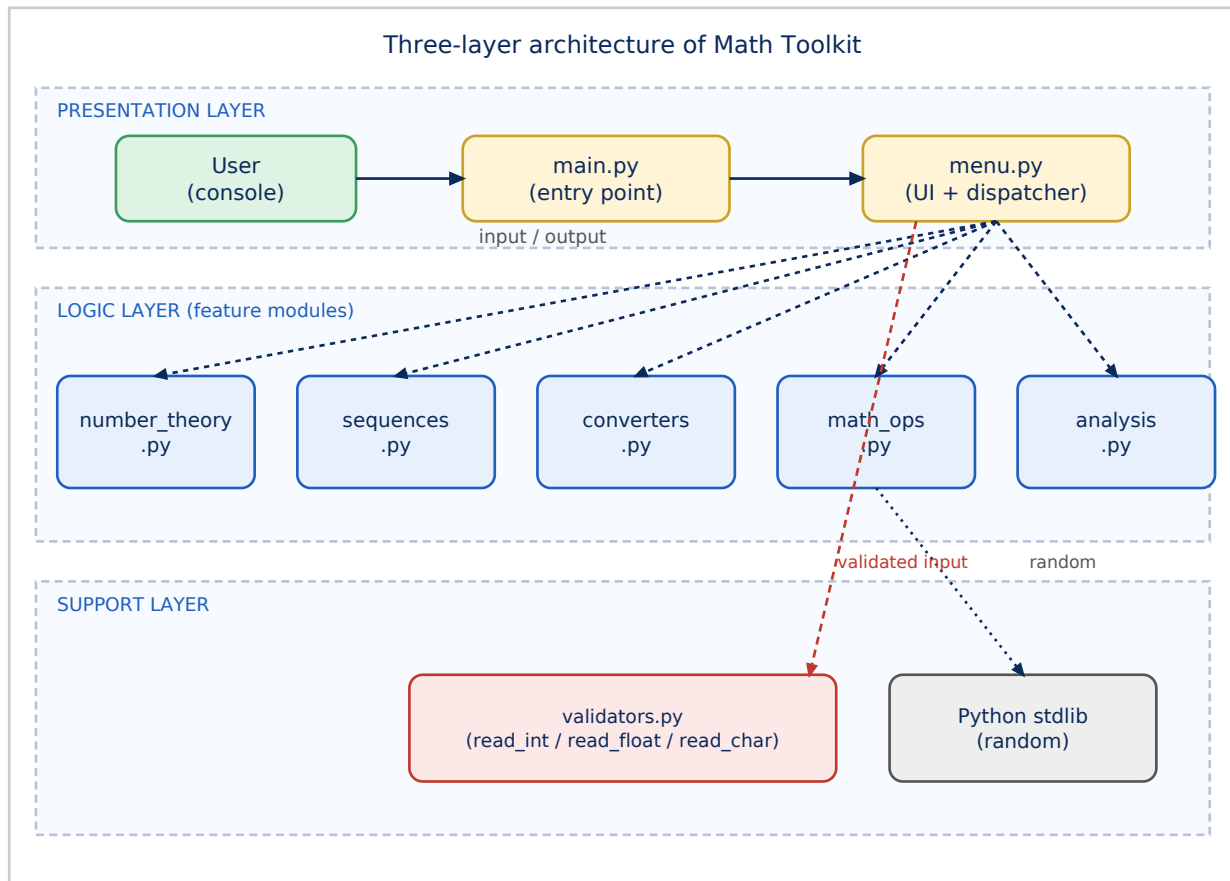


Figure 1: System architecture diagram

Module / File	Responsibility
main.py	Entry point - starts the menu loop.
toolkit/menu.py	Shows the menu, reads the choice, calls the right function, prints results.
toolkit/number_theory.py	is_prime, find_factors, prime_factors, find_gcd, smallest_divisor.
toolkit/sequences.py	find_factorial, fibonacci.
toolkit/converters.py	reverse_num, dec_to_bin, char_to_number.
toolkit/math_ops.py	square_root, power, random_number.
toolkit/analysis.py	full_analysis - combines number_theory and converters.
toolkit/validators.py	read_int, read_float, read_char - safe input with re-prompting.
tests/test_toolkit.py	17 unit tests for all modules.

6. Design Diagrams

6.1 Use Case Diagram

There is one actor - the Student/User. Every feature group is a use case; input validation is included in every one of them.

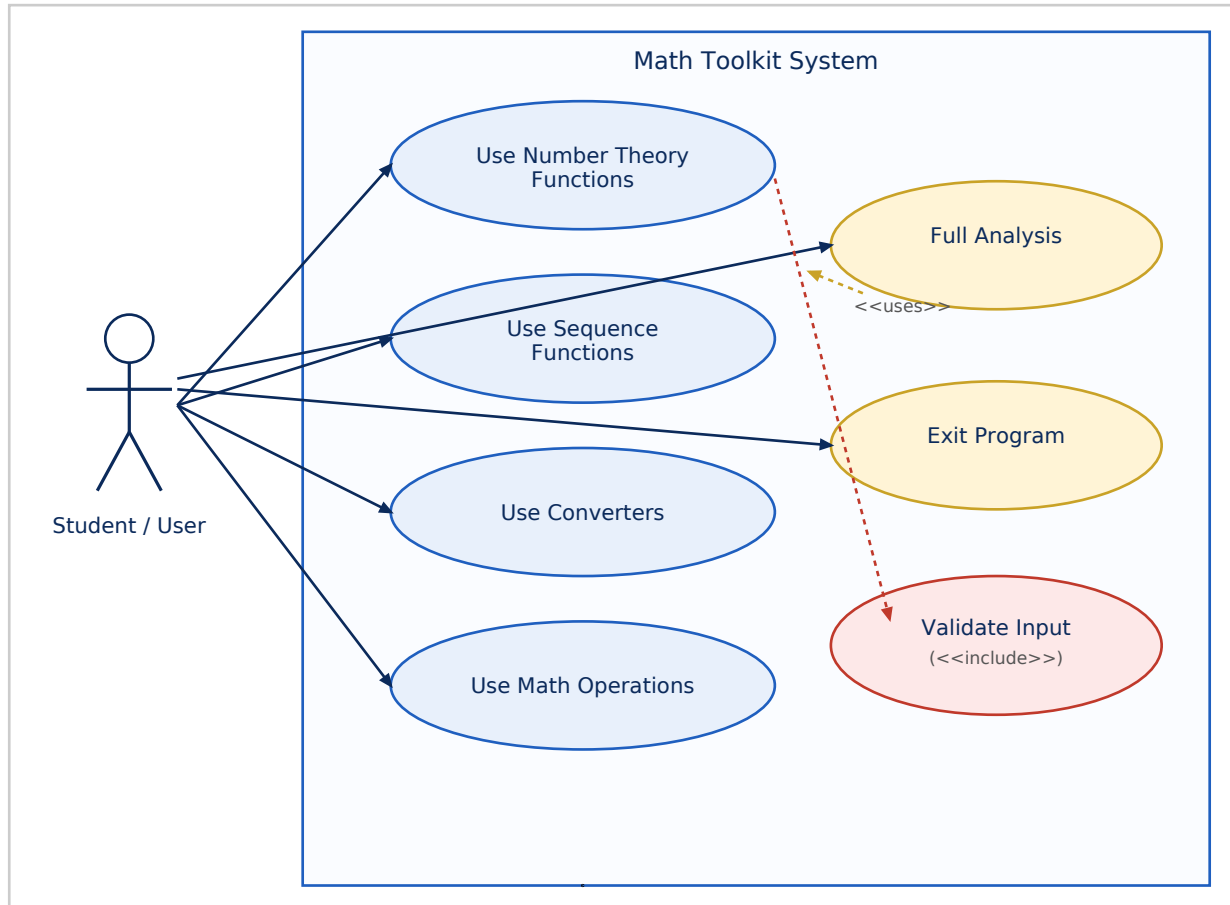


Figure 2: Use case diagram

6.2 Workflow Diagram

The program runs in a loop: show menu, read and validate the choice, collect validated inputs, call the function, show the result and return to the menu until the user selects option 15.

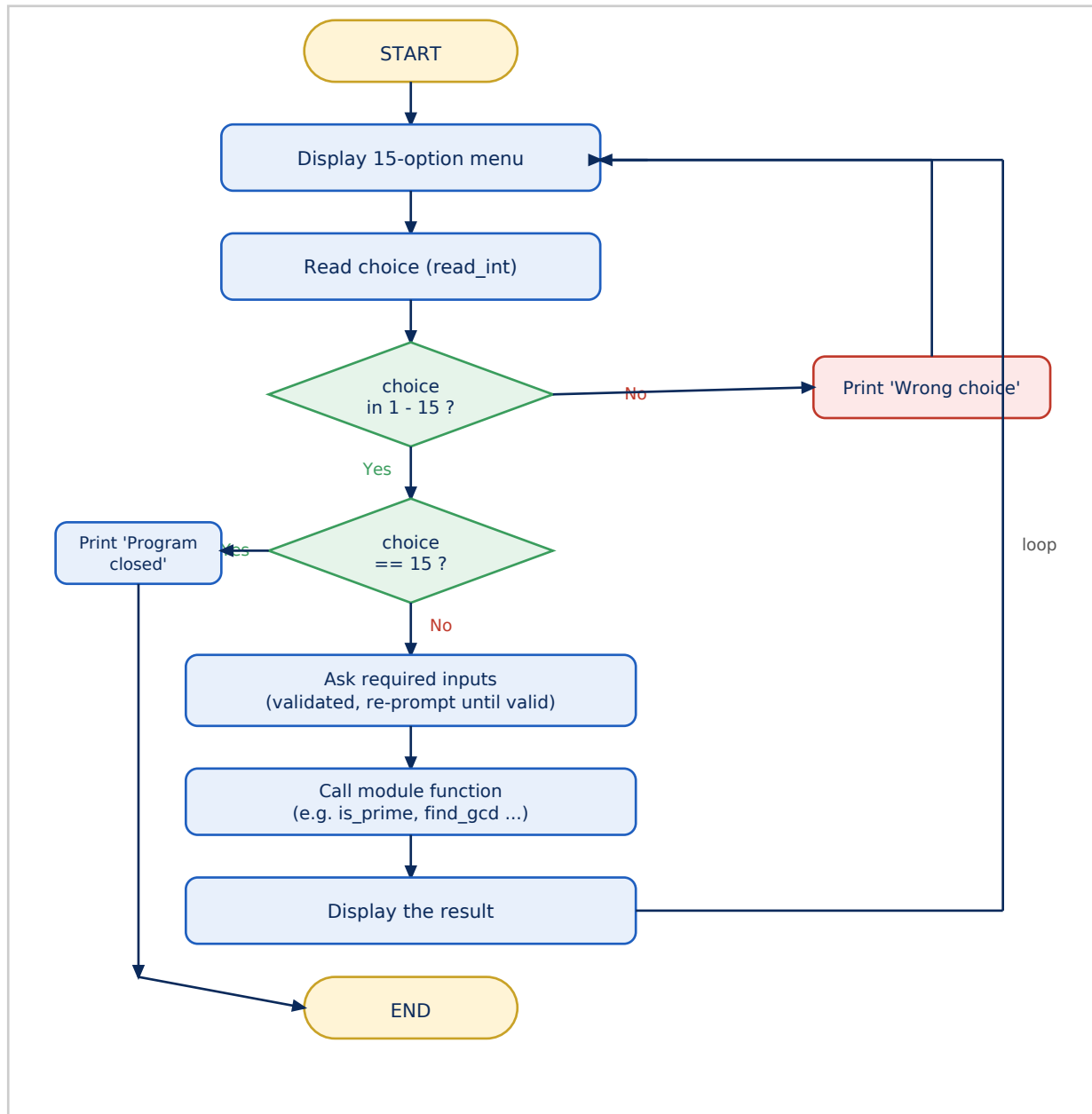


Figure 3: Workflow (flowchart) of the application

6.3 Sequence Diagram

Scenario: the user selects Check Prime, first types an invalid value ('abc'), is asked again, then enters 97.

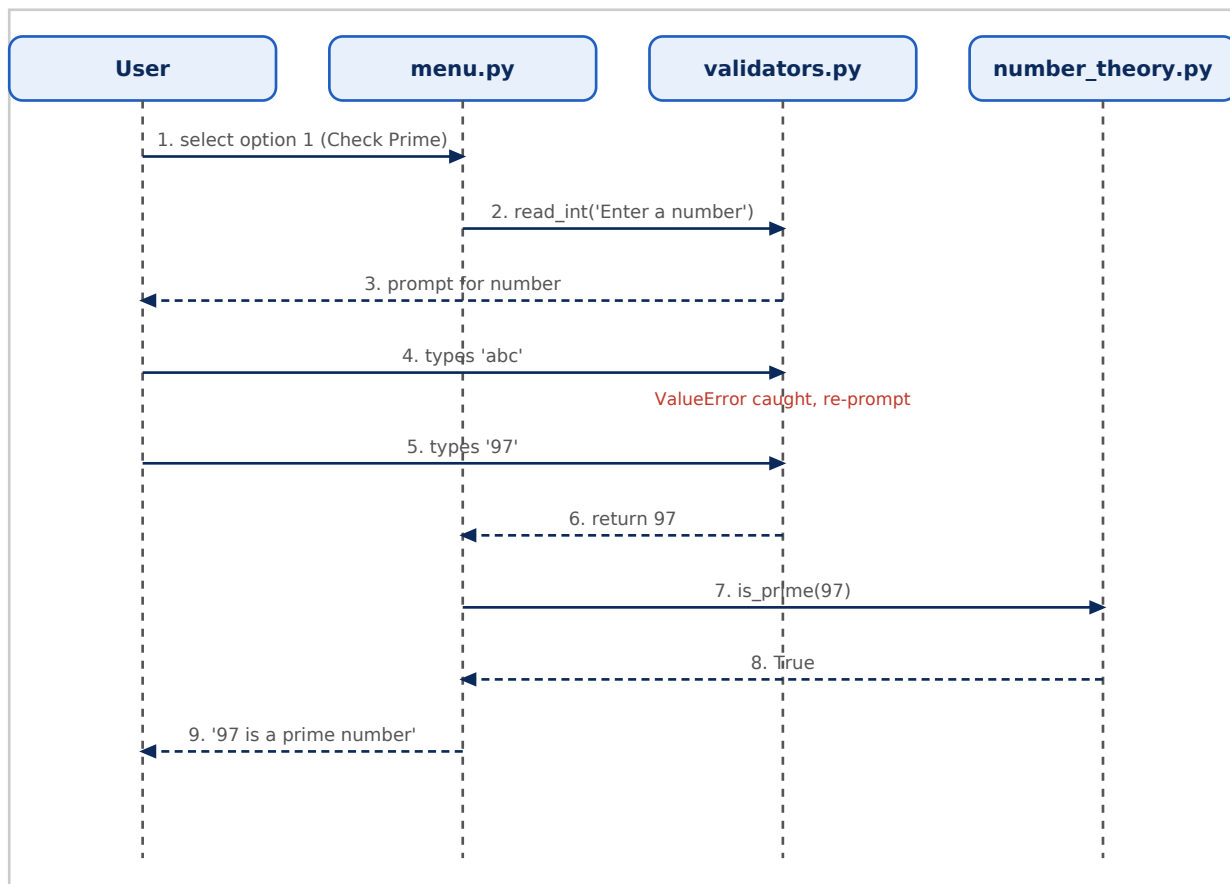


Figure 4: Sequence diagram - Check Prime with input validation

6.4 Class / Component Diagram

The project uses a procedural, function-based design (as taught in this course), so the diagram shows **components (modules)** with their public functions and their dependencies.

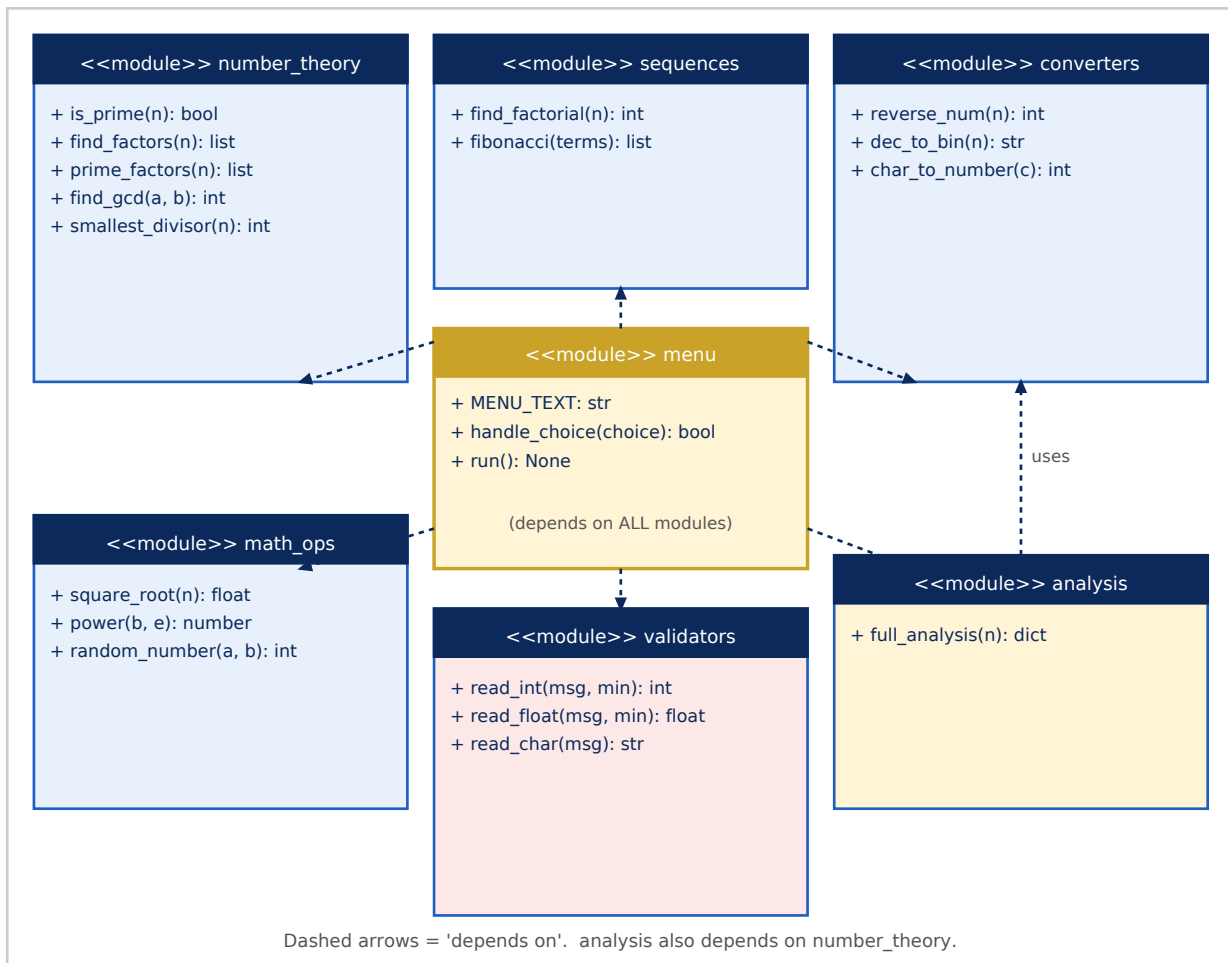


Figure 5: Component diagram

6.5 ER Diagram

Not applicable. Math Toolkit does not store any data - all calculations are done in memory and no database or file storage is used, so no ER diagram or schema is required.

7. Design Decisions & Rationale

Decision	Rationale
Split one big script into 7 modules	The first version was a single file. Splitting by topic makes code easier to find, test and extend, and meets the 5-10 module requirement.
Functions never call input() or print()	Pure functions (input -> return value) are easy to unit test. Only menu.py and validators.py talk to the user.
Separate validators.py	One place for all input checking avoids repeating try/except blocks in 14 menu branches.
Loops-and-conditions style algorithms	Keeps the code in line with course concepts (loops, lists, while) instead of hiding logic in library shortcuts.
Prime check up to square root of n	If n has a divisor larger than its square root, it also has a smaller one - so checking up to $\sqrt{n}$ is enough and much faster than checking up to $n-1$.
Euclidean algorithm for GCD	Faster and simpler than testing every possible divisor.
Dictionary result for Full Analysis	full_analysis returns a dict, so it can be tested; menu.py only formats the output.
Console interface	Suits the course level; keeps focus on logic rather than GUI libraries.
Standard library only	No installation problems; runs anywhere Python runs.

8. Implementation Details

The complete source code is on GitHub (see References). Important extracts are shown below.

8.1 Prime check (number_theory.py)

```
def is_prime(n):
    if n < 2:
        return False
    # Only need to check up to the square root of n
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

8.2 Euclidean GCD (number_theory.py)

```
def find_gcd(a, b):
    while b != 0:
        rem = a % b
        a = b
        b = rem
    return a
```

8.3 Safe input (validators.py)

```
def read_int(message, minimum=None):
    while True:
        try:
            value = int(input(message))
        except ValueError:
            print("Invalid input! Please enter a whole number.")
            continue
        if minimum is not None and value < minimum:
            print("Number must be", minimum, "or greater.")
            continue
        return value
```

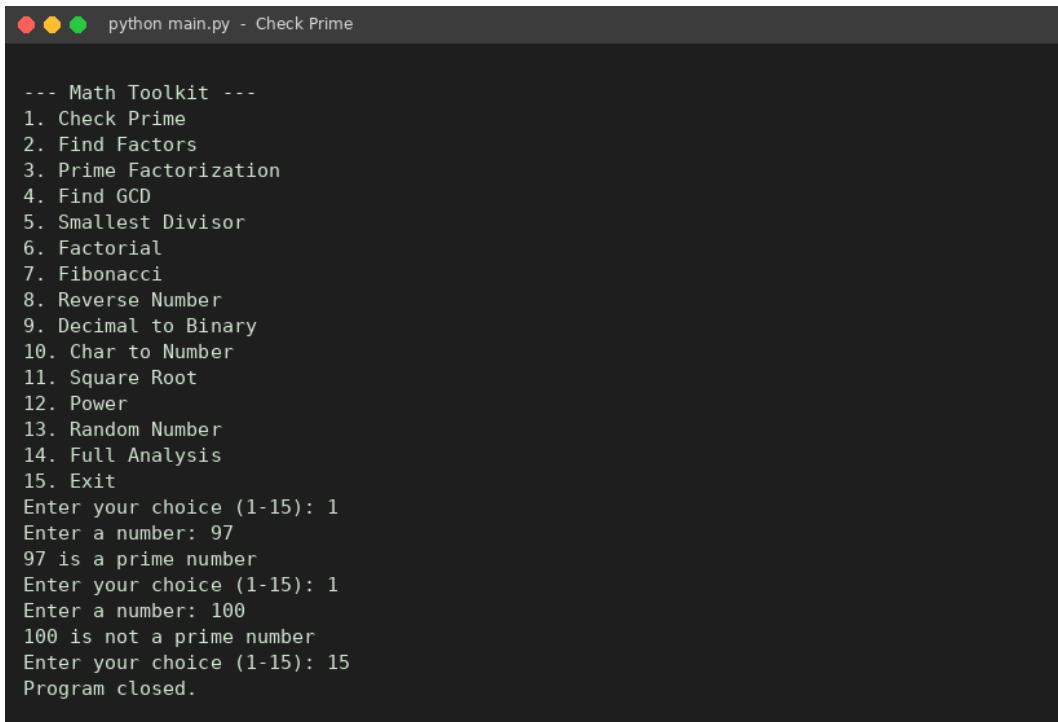
8.4 Course concepts used

Concept	Where it is used
Variables & data types	int, float, str, list, dict throughout the modules
Conditional statements	if / elif / else in menu.py, is_prime, validators
Loops (for, while)	factors, Fibonacci, GCD, reverse, binary conversion, menu loop
Functions & modularity	23 functions across 7 modules
Lists & strings	factors list, Fibonacci list, binary string building
Exception handling	try / except ValueError in validators.py
Operators	%, //, **, comparison and logical operators

8.5 Version control

The project is maintained in a Git repository. Suggested commit history: initial script -> split into modules -> add validators -> add unit tests -> add README, statement.md and report.

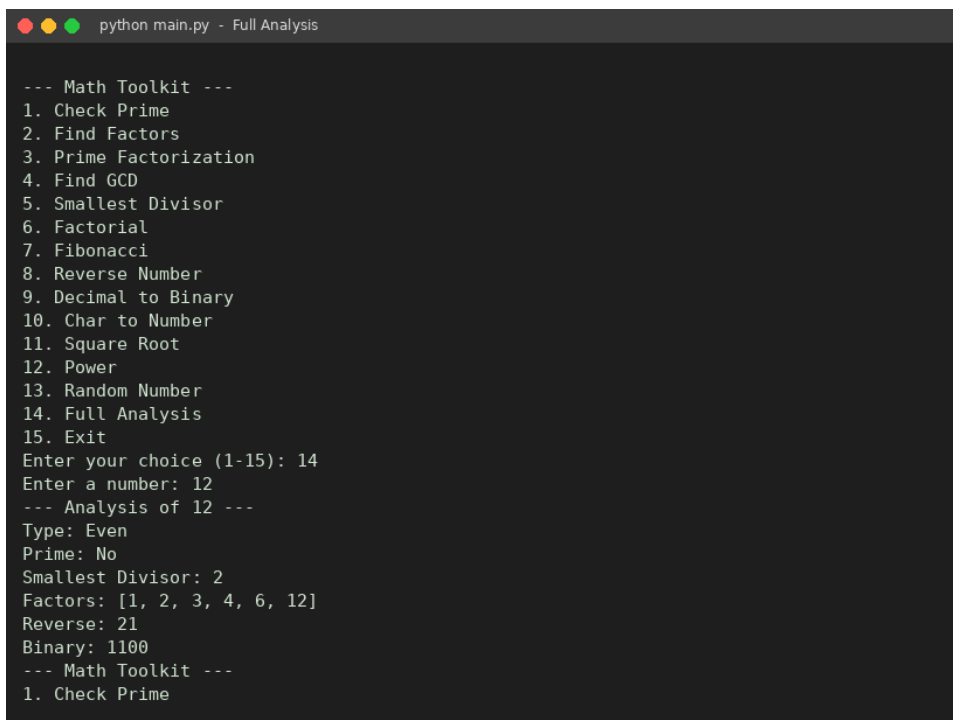
9. Screenshots / Results



```
python main.py - Check Prime

--- Math Toolkit ---
1. Check Prime
2. Find Factors
3. Prime Factorization
4. Find GCD
5. Smallest Divisor
6. Factorial
7. Fibonacci
8. Reverse Number
9. Decimal to Binary
10. Char to Number
11. Square Root
12. Power
13. Random Number
14. Full Analysis
15. Exit
Enter your choice (1-15): 1
Enter a number: 97
97 is a prime number
Enter your choice (1-15): 1
Enter a number: 100
100 is not a prime number
Enter your choice (1-15): 15
Program closed.
```

Figure 6: Menu and Check Prime (97 is prime, 100 is not)



```
python main.py - Full Analysis

--- Math Toolkit ---
1. Check Prime
2. Find Factors
3. Prime Factorization
4. Find GCD
5. Smallest Divisor
6. Factorial
7. Fibonacci
8. Reverse Number
9. Decimal to Binary
10. Char to Number
11. Square Root
12. Power
13. Random Number
14. Full Analysis
15. Exit
Enter your choice (1-15): 14
Enter a number: 12
--- Analysis of 12 ---
Type: Even
Prime: No
Smallest Divisor: 2
Factors: [1, 2, 3, 4, 6, 12]
Reverse: 21
Binary: 1100
--- Math Toolkit ---
1. Check Prime
```

Figure 7: Full Analysis of the number 12

```
python main.py - Validation & Error Handling

Enter your choice (1-15): abc
Invalid input! Please enter a whole number.
Enter your choice (1-15): 99
Wrong choice! Please enter between 1 to 15.
Enter your choice (1-15): 6
Enter a number: -3
Number must be 0 or greater.
Enter a number: 0
Factorial is: 1
Enter your choice (1-15): 5
Enter a number: 15
Smallest divisor is: 3
```

Figure 8: Input validation - invalid text, out-of-range choice and negative factorial input handled

10. Testing Approach

Testing is done with Python's built-in **unittest** module. Functions are pure, so they are tested directly; keyboard input in validators is simulated with `unittest.mock.patch`. Tests cover normal values, boundary values (0, 1, negative numbers) and invalid input.

Test class	Tests	What is verified
TestNumberTheory	5	primes (2, 97, 1, 0, -7, 100), factors, prime factorization, GCD, smallest divisor
TestSequences	2	factorial of 0 and 5; Fibonacci of 7 and 0 terms
TestConverters	3	reverse 1234 -> 4321; binary 10 -> 1010, 0 -> '0'; char 'A' -> 65
TestMathOps	2	sqrt(16), 2^10, random number always inside range (50 trials)
TestAnalysis	1	complete analysis report of 12
TestValidators	4	re-prompting on 'abc', "", negative, wrong-length character

```
python -m unittest discover tests -v

Invalid input! Please enter a number.
Number must be 0 or greater.
Invalid input! Please enter a whole number.
Invalid input! Please enter a whole number.
test_full_analysis (test_toolkit.TestAnalysis.test_full_analysis) ... ok
test_binary (test_toolkit.TestConverters.test_binary) ... ok
test_char (test_toolkit.TestConverters.test_char) ... ok
test_reverse (test_toolkit.TestConverters.test_reverse) ... ok
test_random_in_range (test_toolkit.TestMathOps.test_random_in_range) ... ok
test_root_and_power (test_toolkit.TestMathOps.test_root_and_power) ... ok
test_find_factors (test_toolkit.TestNumberTheory.test_find_factors) ... ok
test_gcd (test_toolkit.TestNumberTheory.test_gcd) ... ok
test_is_prime (test_toolkit.TestNumberTheory.test_is_prime) ... ok
test_prime_factors (test_toolkit.TestNumberTheory.test_prime_factors) ... ok
test_smallest_divisor (test_toolkit.TestNumberTheory.test_smallest_divisor) ... ok
test_factorial (test_toolkit.TestSequences.test_factorial) ... ok
test_fibonacci (test_toolkit.TestSequences.test_fibonacci) ... ok
test_read_char (test_toolkit.TestValidators.test_read_char) ... ok
test_read_float (test_toolkit.TestValidators.test_read_float) ... ok
test_read_int_minimum (test_toolkit.TestValidators.test_read_int_minimum) ... ok
test_read_int_retries (test_toolkit.TestValidators.test_read_int_retries) ... ok

-----
Ran 17 tests in 0.002s

OK
```

Figure 9: All 17 unit tests pass

Result: 17 tests run, 17 passed, 0 failed. Manual testing of the menu was also done for all 15 options.

11. Challenges Faced

- **Program crashed on wrong input.** Typing a letter instead of a number caused a `ValueError`. Solved by creating `validators.py` with `try/except` and re-prompting.
- **Negative and zero inputs.** Factorial, reverse and binary functions gave wrong results for negative numbers. Solved by setting a minimum value in `read_int()`.
- **Slow prime check.** Checking every number up to $n-1$ is slow for big values. Solved by stopping at the square root of n .
- **Splitting one file into many.** Deciding which function belongs where, and avoiding circular imports, needed planning. Solved by grouping functions by topic and making `menu.py` the only module that imports from all others.
- **Testing code that uses `input()`.** Solved by separating logic from input/output and using `mock.patch` for the validators.
- **Empty results.** `smallest_divisor(n)` originally returned nothing for some inputs; now it returns n itself for primes and handles $n < 2$ explicitly.

12. Learnings & Key Takeaways

- Small, single-purpose functions are easier to write, test and reuse.
- Separating logic from input/output makes programs testable.
- Input validation is not optional - real users type unexpected things.
- A better algorithm (sqrt limit, Euclid) can matter more than faster hardware.
- Version control, documentation and tests are part of programming, not extras.
- A working project needs planning: requirements -> design -> code -> test -> document.

13. Future Enhancements

- Graphical interface using Tkinter.
- History log file that saves previous calculations (would add file storage and an ER/schema design).
- More tools: LCM, perfect numbers, Armstrong numbers, palindromes, number-to-words, base conversion for octal and hexadecimal.
- Support for very large numbers with faster algorithms (Sieve of Eratosthenes).
- Package the project so it can be installed with pip; add continuous testing with GitHub Actions.

14. References

- Python Software Foundation - Python 3 Documentation: <https://docs.python.org/3/>
- Python unittest - Unit testing framework: <https://docs.python.org/3/library/unittest.html>
- VITYarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.
- Euclidean algorithm for GCD - standard number theory reference.
- Course material: Introduction to Problem Solving and Programming, VIT Bhopal University.
- GitHub repository: <https://github.com/<your-username>/math-toolkit> (add your link here).